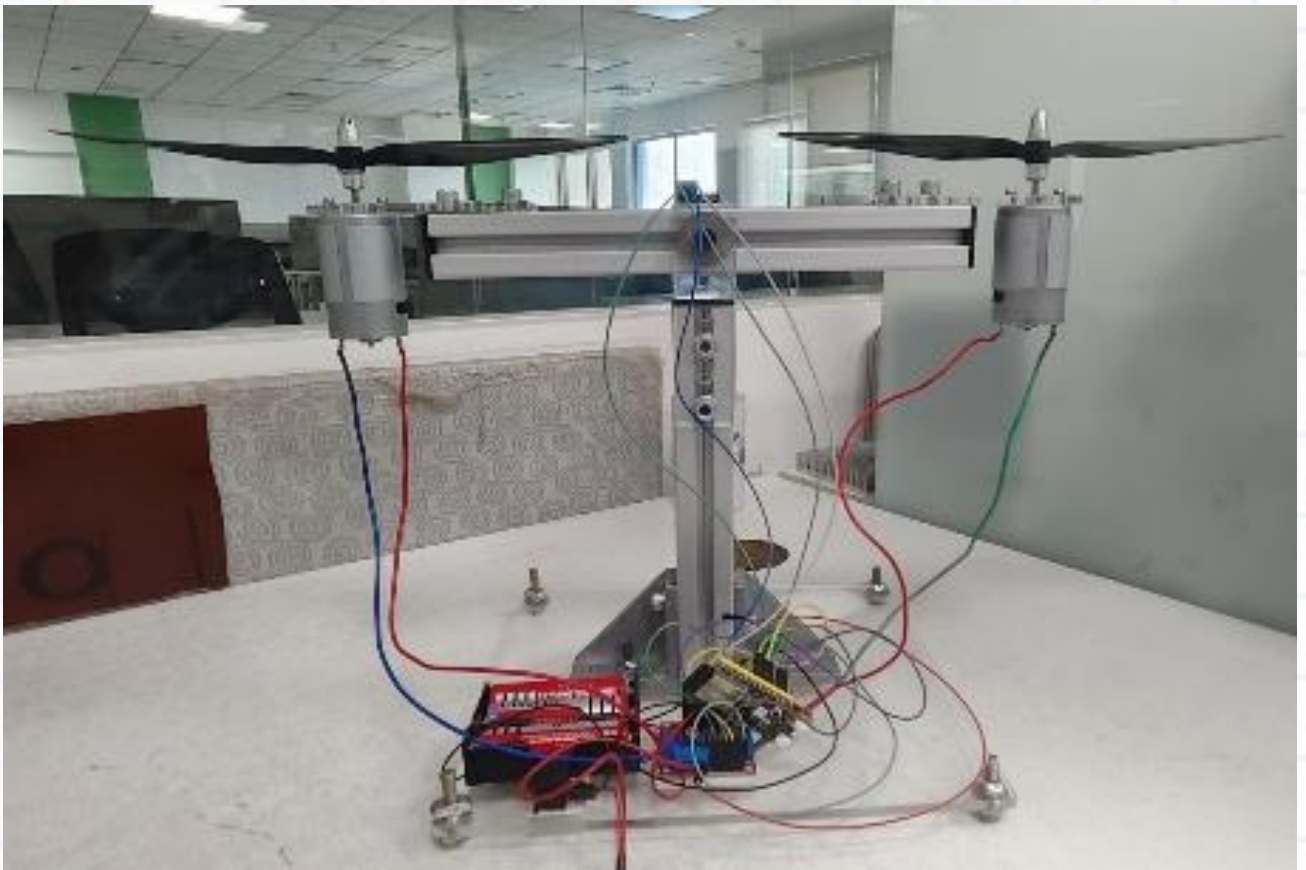


# “Pitch Perfect – PID Control”

(Pitch stabilisation algorithm for two vertical rotor setup)



## Index

Outcome

Components

Concept

Hardware Connections in detail

Software Implementing the project

Tasks

## Outcome:

To design and implement a **one-dimensional pitch stabilization system** using a **Proportional-Integral-Derivative (PID) control algorithm**, enabling accurate real-time control of the pitch angle through continuous sensor feedback and actuator adjustments for dynamic motion stability.

## Concept:

This project revolves around the development of a one-dimensional pitch stabilization system using a Proportional-Integral-Derivative (PID) control algorithm to maintain a desired pitch angle in real-time. The primary objective is to dynamically control the orientation of a platform or beam, ensuring it remains stable even under varying conditions or external disturbances.

The system continuously measures the pitch angle using an inertial sensor such as an IMU (Inertial Measurement Unit) or gyroscope. A microcontroller processes this sensor data and implements the PID algorithm to calculate the necessary correction to maintain the desired pitch. The output of the controller is then used to drive an actuator (such as a DC motor), which adjusts the physical orientation of the system to reduce pitch error.

This project highlights the principles of closed-loop feedback control and demonstrates how PID algorithms can be used to stabilize dynamic systems effectively. It provides hands-on experience with key engineering concepts such as real-time sensor integration, actuator control, and tuning of PID parameters. This makes it highly relevant in fields like robotics, autonomous systems, flight control, and mechatronics engineering, where maintaining precise orientation and balance is crucial.



## PID (Proportional-Integral-Derivative) Controller

A PID controller is a type of feedback control system commonly used in industrial and automation applications to maintain the desired output of a system by minimizing the error between a setpoint and the measured process variable.

The PID controller works by calculating three components:

- Proportional (P): Responds to the current error.
- Integral (I): Responds to the accumulation of past errors.
- Derivative (D): Predicts future error based on the current rate of change.

By tuning these three parameters, the controller can provide smooth, stable, and accurate control for dynamic systems. PID controllers are widely used in applications like temperature control, speed regulation, robotics, and balancing systems due to their simplicity and effectiveness.

For Detail theory use the below table

| Topic               | Source               |
|---------------------|----------------------|
| Introduction to PID | <a href="#">Link</a> |
| Basics of PID       | <a href="#">Link</a> |
| Working principles  | <a href="#">Link</a> |
| Applications        | <a href="#">Link</a> |

## Introduction

The Boeing CH-47 Chinook is a tandem-rotor helicopter developed and manufactured by the American rotorcraft company Vertol. The Chinook is a heavy-lift helicopter and is one of the most heavy-lift western helicopters. Its name, Chinook, comes from the Chinook Indians of Oregon and Washington.



**Figure 1: Boeing CH-47 Chinook**

The rotors of the helicopter setup rotate in the opposite direction, this cancels the counter-rotation, therefore, no tail rotor is necessary to stabilise the yaw.

The given project focuses on building a simplified simulator of a Chinook helicopter that is supported at the centre of mass but is free to pivot, pitch up, and down.

## Components

| Sno | Components with specification                                                                                 | Quantity  |
|-----|---------------------------------------------------------------------------------------------------------------|-----------|
| 1.  | ESP32 C6 Dev module                                                                                           | 1         |
| 2.  | USB to Type – C cable                                                                                         | 1         |
| 4.  | L298N Motor driver                                                                                            | 1         |
| 5.  | RS – 555 DC motor                                                                                             | 2         |
| 6.  | 3.7V 5000 Mah battery                                                                                         | 3         |
| 7.  | 3X18650 battery holder with switch                                                                            | 1         |
| 8.  | Jumper wires                                                                                                  | As needed |
| 9.  | Drone propeller                                                                                               | 2         |
| 10. | IMU MPU6050 sensor                                                                                            | 1         |
| 11. | 20x20 Aluminium Extrusion<br>a. 30X30 200mm length – 1<br>b. 30X30 254mm length – 1<br>c. L angle bracket - 2 | 2         |
| 12. | 5mm Acrylic sheet                                                                                             | 1         |
| 13. | Fastener & Accessories                                                                                        | As needed |



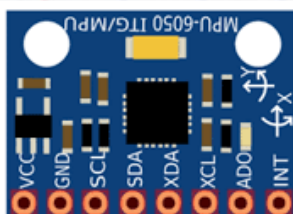
ESP32 C-6 Module



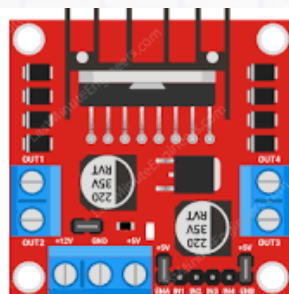
RS – 555 DC Motors



Drone Propellers



MPU6050 Sensor



L298N Motor Driver



3.7V 5000 Mah battery







Rocker Switch



USB to Type – C cable



Jumper Wires



Mechanical Setup

**Note:** We used Aluminium exclusion and Acrylic Sheet for pitch perfect set up.

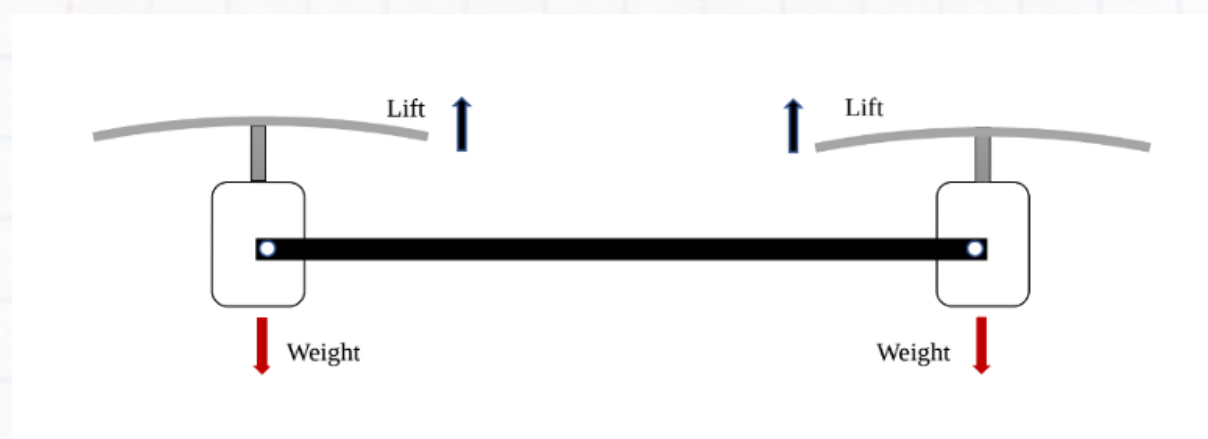
### Install Required Libraries

- Go to Sketch > Include Library > Manage Libraries.
- Connect Your ESP32 C-6 Module
- Connect your ESP32 Module to your computer using a USB cable.
- Go to Tools > Board > ESP32C6 Dev Module.
- Go to Tools > Port and select the COM port to which your ESP32C-6 is connected.
- Go to sketch > Include Library > Manage Libraries.
- Search MPU6050\_light and install **MPU6050\_light** by **refetick**.

### Component placement

An **aluminium extrusion block** of 400 mm is mounted on top of a pivot. Two motors are installed on the extrusion using acrylic sheet, rotating in opposite directions and fixed symmetrically along the centre line. These motors are mounted 65 mm from the top surface of the aluminium block. The extrusion has an approximate width of 30 mm and is pivoted at a height of 125 mm and 20 mm from the top. It is free to move over the pivot, which is secured using a 6 mm bolt of 60 mm length. The MPU6050 sensor is mounted on the top of the aluminium extrusion, directly above the central pivot point and the connections are made according to the circuit diagram. The connections are made sturdy by using tape wherever needed.

## Methodology



**Figure 2: Free Body Diagram**

### Design and build the physical configuration:

The first step in this project was to design and build a simplified physical configuration of the Chinook helicopter. The setup consists of a small-scale helicopter model with two vertical propellers mounted at each end. The model is supported at the centre of mass but can rotate freely (tilt up or down). The length of the helicopter fuselage is limited to 400 mm. The motors are controlled by a microcontroller.



**Implement the control algorithm:**

The next step is to implement a control algorithm that can balance the helicopter's altitude by controlling the lift on each propeller. The control algorithm uses MPU6050 Sensor, having an accelerometer and gyroscope sensor to measure the helicopter's tilt angle and adjust the lift of each propeller accordingly. A PID controller can be used for this purpose. ESP32- C6 was used as a microcontroller in the given project.

**Check the control algorithm:**

Once the physical configuration and control algorithm have been built, the next step is to check the helicopter's ability to balance the altitude according to the algorithm. The helicopter must be tested for balance when an external load is applied to push the CG out of the pivot point. The load should be reasonable and depend on the torque of the motor.

**Collect data and analyse results:**

During the test phase, data is collected on the angle of inclination, lift per rotor, and applied external loads. The data gets analysed to determine how well the control algorithm is performing and to identify areas for improvement.

**Fine-tuning the control algorithm:**

Based on the test results and data analysis, the control algorithm can be fine-tuned to improve its performance. This involves adjusting the parameters of the PID controller or adding additional sensors or control functions. Here we adjust the PID constant values. Once the control algorithm was fine-tuned, a final series of tests were performed to ensure that it works as expected.

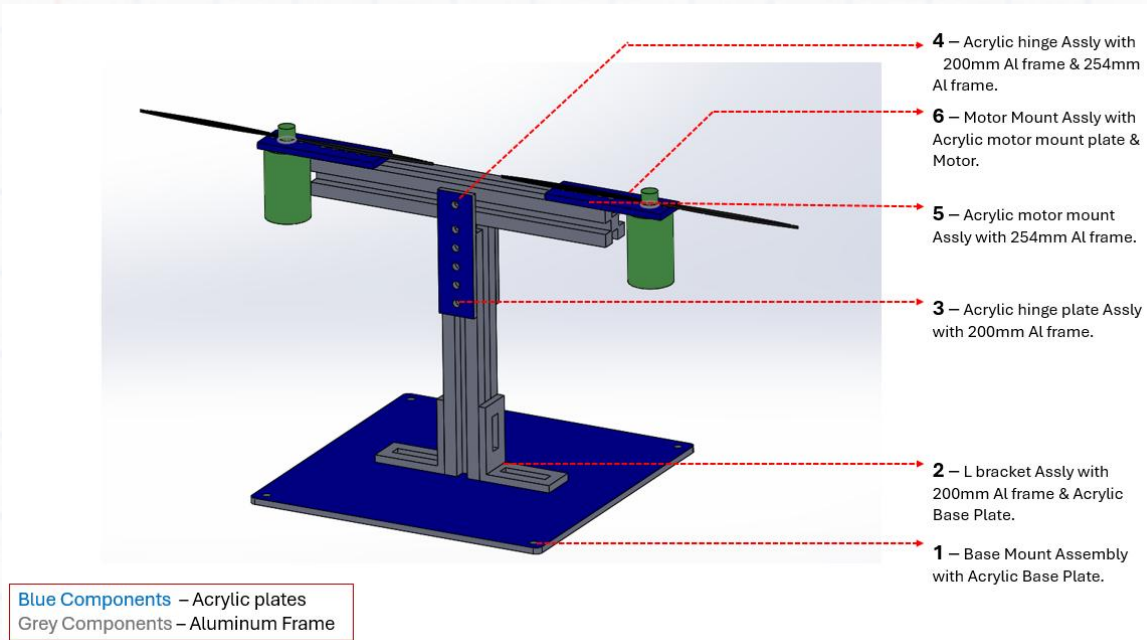
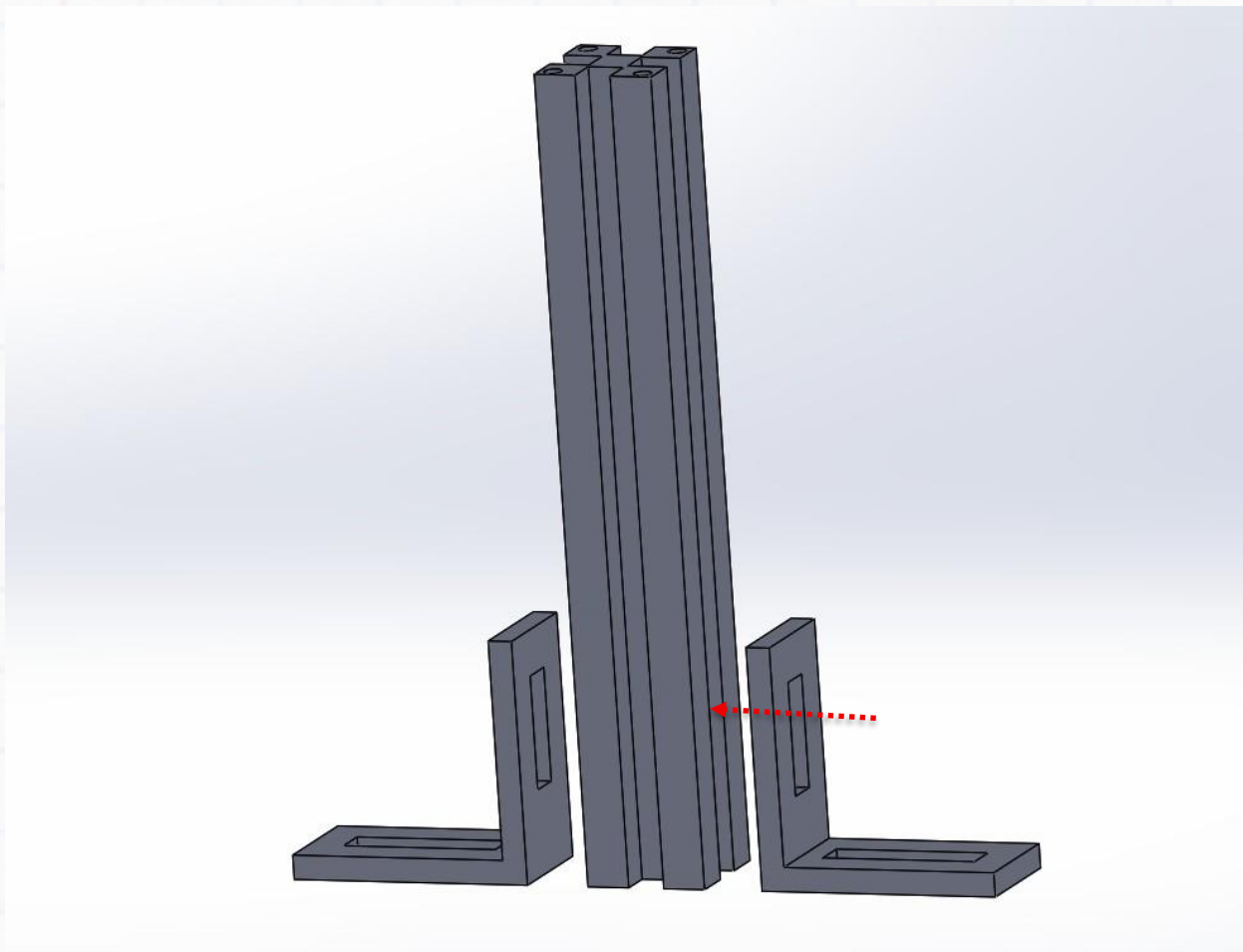
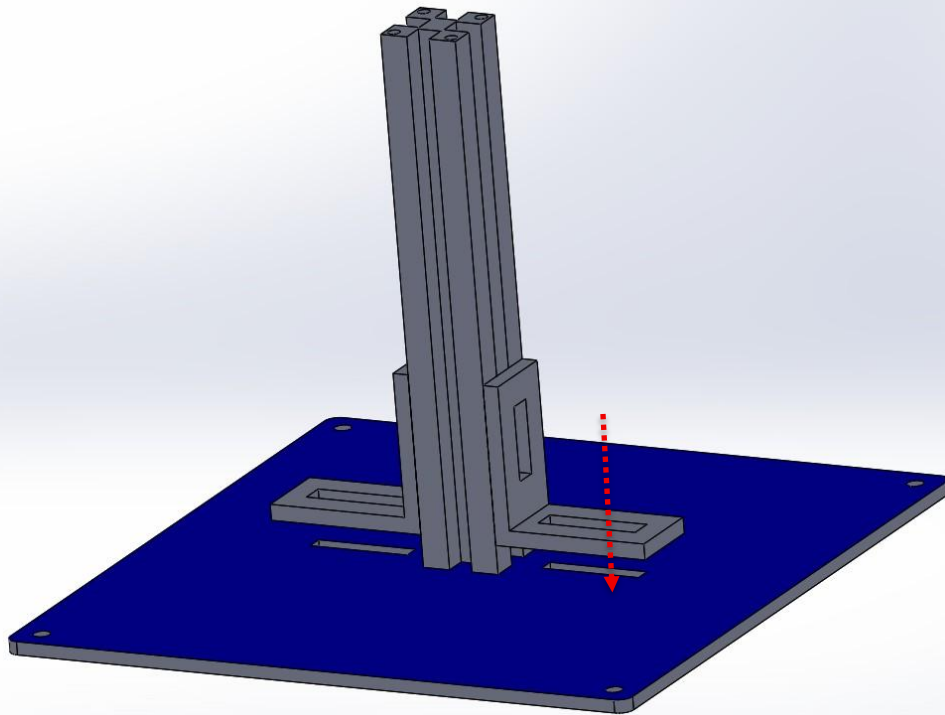


Figure 3: Project Setup

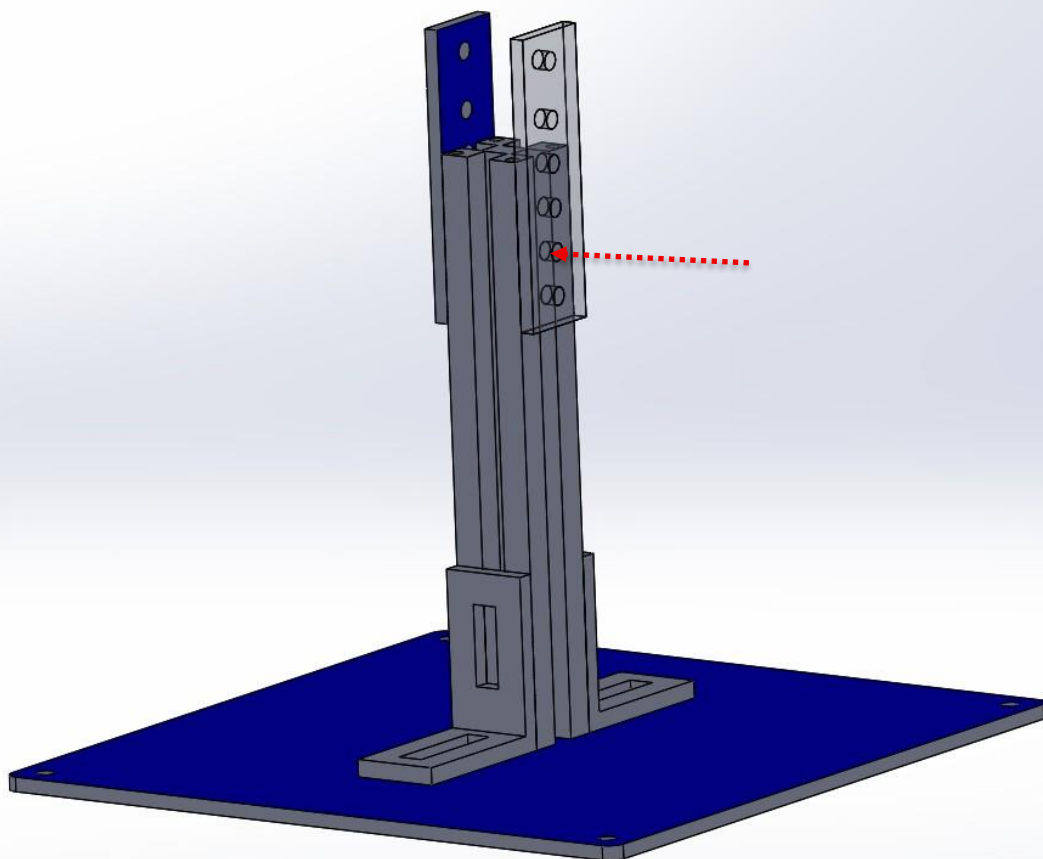
## Assembly Steps



**Step 1** – Assemble 200mm Aluminium frame with L brackets on either side

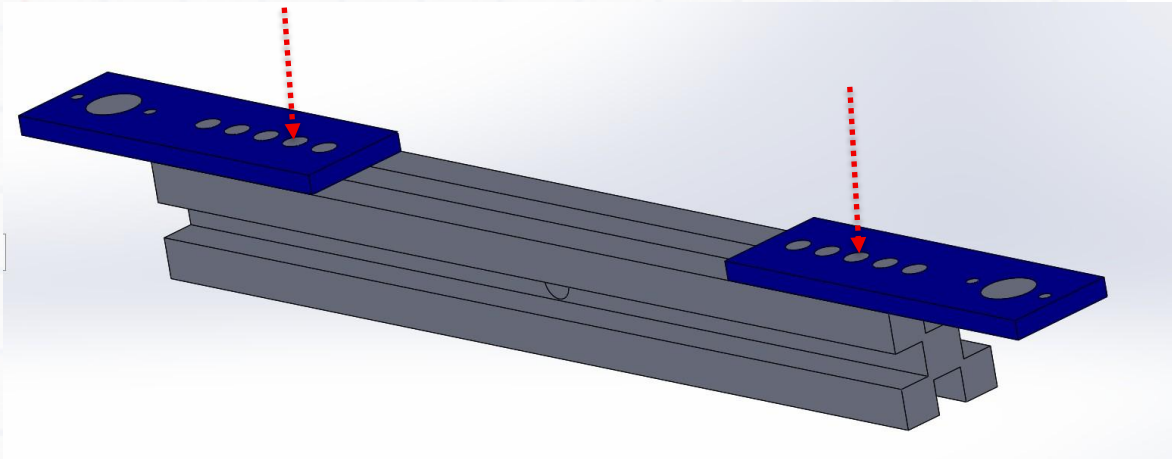


**Step 2** – Assemble L bracket to the acrylic sheet on either side

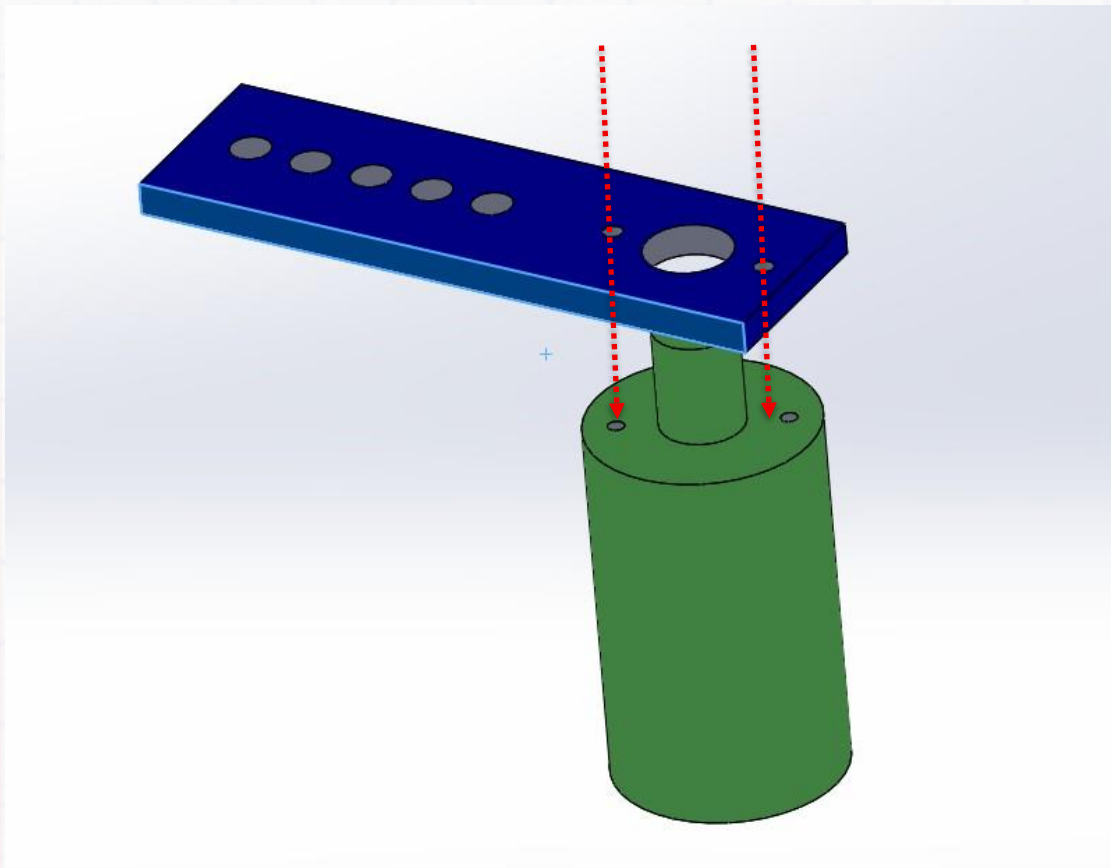


**Step 3** – Mount acrylic hinge plate to the top side of aluminium frame(200mm)

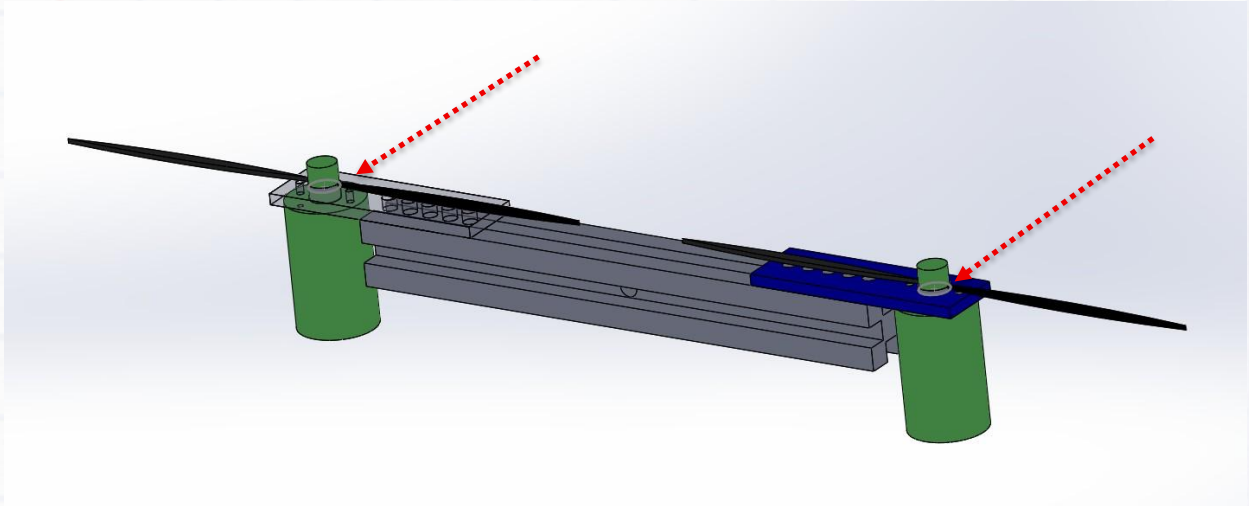




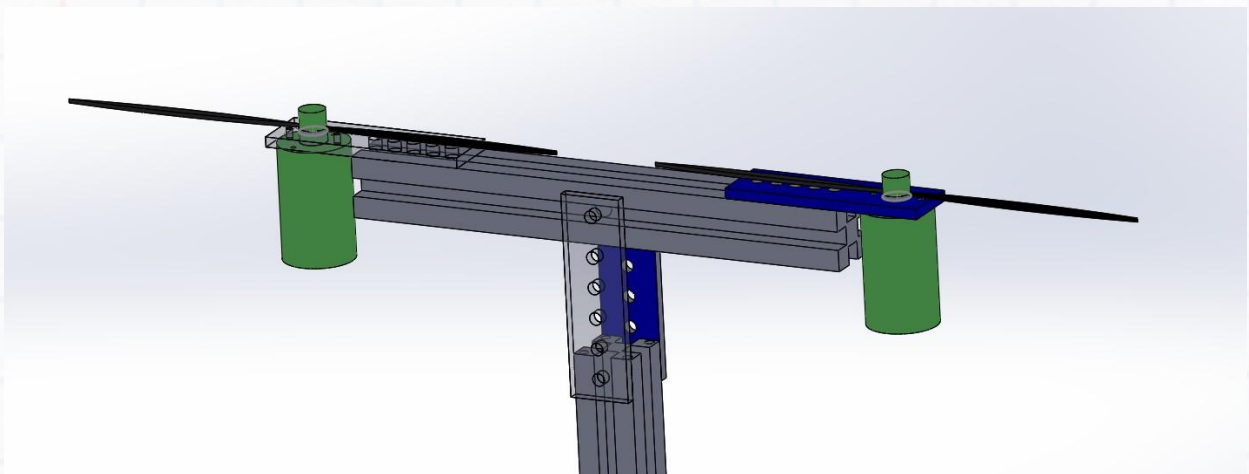
**Step 4** – Assemble acrylic hinge plate to the top side of Aluminium frame(254mm)



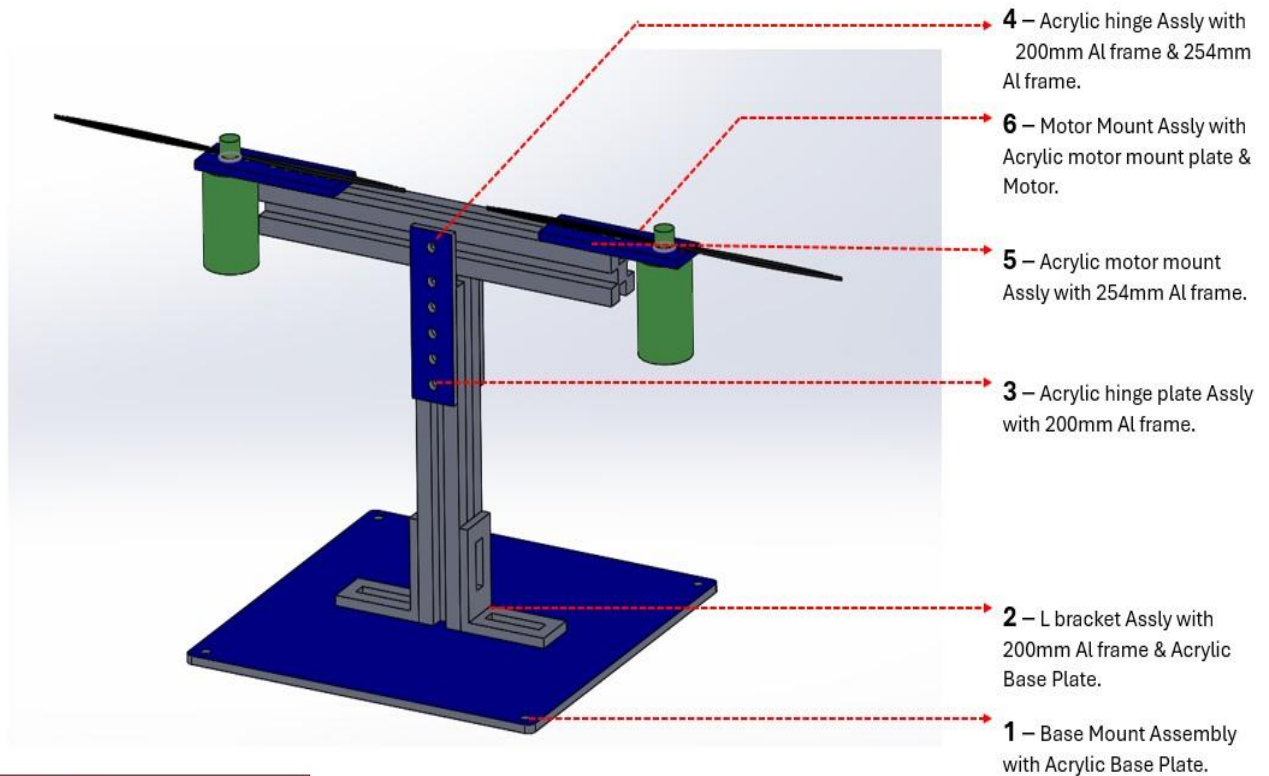
**Step 5** – Mount Motor to the Motor Mount Acrylic plates



**Step 6 – Mount Propeller to the Motor Shaft and lock them**



**Step 7 – Assembled view of the top portion**



Blue Components – Acrylic plates  
Grey Components – Aluminum Frame

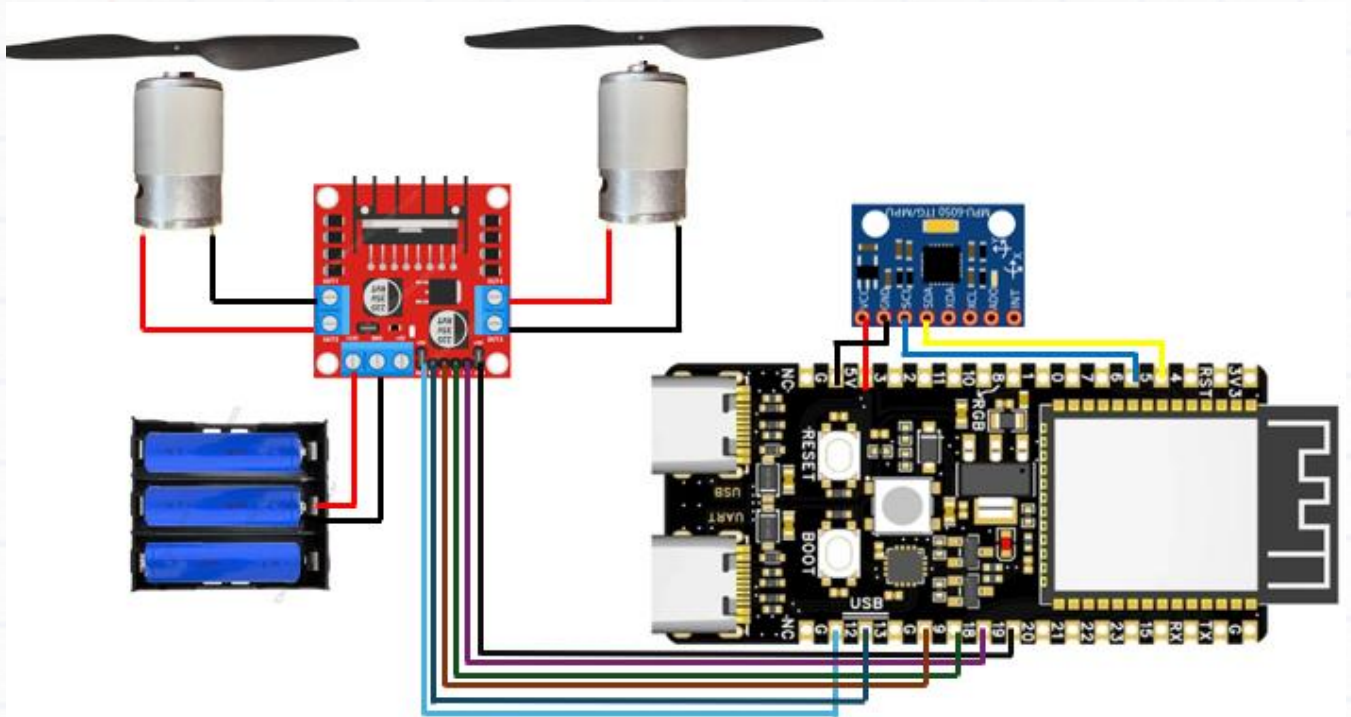
### Completed Assembly

The length of the aluminium extrusion block mounted as the body is 400mm. The motors are fixed at both sides of this block at 20mm from the two edges as shown, and the two motors are mounted with propellers. The pivot position at the centre corresponds to the CG of the body. The total range of motion allowed by the design is  $\pm 25$  degrees. The motion is restricted by another aluminium extrusion such that it does not hit the propeller to the ground. The body setup is supported by aluminium extrusion, acrylic sheet using L angle extrusion and T nut (4mm). A 6mm bolt with T nut is inserted into the body at the CG and stuck with the body without any motion to the body, this gives rotational mobility to the system. The motor chosen is of 4000 rpm, and the propellers used are chosen such that thrust given by the motor is quite good for the setup.



## Circuit diagram

**NOTE:** Before starting the connections, verify using a multimeter that all the wires are working. Also ensure that the connections are strong, else the setup may not work.



- GND to GND
- SDA to GPIO 4
- SCL to GPIO 5

(DC Motor to L298N)

- Terminal 1 of the Motor 1 to OUT1
- Terminal 2 of the Motor 1 to OUT2
- Terminal 1 of the Motor 2 to OUT 3
- Terminal 2 of the Motor 2 to OUT 4

(Battery Holder to L298N)

- Positive terminal of battery to 12V terminal
- Negative terminal of battery to GND terminal

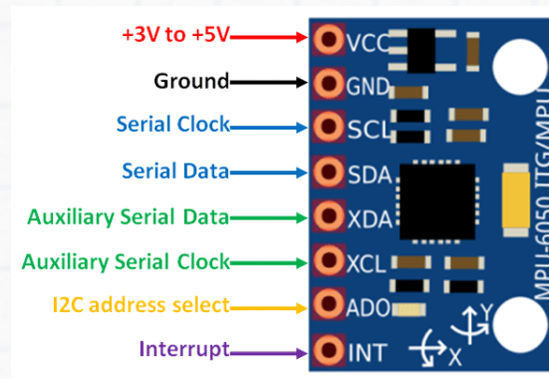
(L298N to ESP32C6)

- ENA to GPIO 12

- IN1 to GPIO 13
- IN2 to GPIO 9
- IN3 to GPIO 18
- IN4 to GPIO 19
- ENB to GPIO 20

## Components in Detail

### 1) MPU6050 Sensor



The **MPU6050** is a widely used 6-axis motion tracking sensor that combines a 3-axis **gyroscope** and a 3-axis **accelerometer** on a single chip. It is designed for applications requiring precise measurement of orientation, motion, and acceleration, such as in drones, self-balancing robots, gesture-controlled devices, and stabilization systems. The sensor works by measuring angular velocity (rate of rotation) through the gyroscope and linear acceleration through the accelerometer. Together, these measurements allow for accurate tracking of pitch, roll, and yaw angles. By using filtering techniques such as the complementary filter or Kalman filter, the data from both sensors can be fused to achieve stable and reliable orientation readings.

The MPU6050 communicates with microcontrollers via the **I<sup>2</sup>C (Inter-Integrated Circuit)** protocol using the SCL (clock) and SDA (data) lines. Its default I2C address is **0x68**, which can be changed to **0x69** by pulling the ADO pin HIGH. It operates on a voltage range of **3.3V to 5V**, making it compatible with most microcontroller boards like Arduino and ESP32C6. The sensor outputs high-resolution motion data, and it also includes a **built-in Digital Motion Processor (DMP)**, which can be used to offload complex calculations from the main microcontroller. The gyroscope can be configured for ranges of  $\pm 250$  to  $\pm 2000$  degrees per second, while the accelerometer

supports  $\pm 2g$  to  $\pm 16g$  ranges. It also features an onboard temperature sensor for internal thermal compensation.

In this project, the MPU6050 plays a critical role by providing **real-time pitch angle measurements**, which serve as the feedback input to the PID controller. The controller processes this data to determine the necessary corrections and commands the actuator (such as a DC motor) to maintain or restore the desired pitch angle. With its small size, low power consumption, and accurate sensing capabilities, the MPU6050 is a fundamental component in implementing closed-loop pitch control systems. It is highly suitable for projects involving **robotics, automation, stabilization, and motion tracking**, offering practical insight into control theory and sensor-based feedback systems.

## 2) RS-555 DC Motor



The **RS-555** is a high-speed brushed DC motor commonly used in robotics, electric vehicles, and industrial automation projects. Known for its balance between torque and speed, this motor is widely used in applications such as fans, small conveyor belts, robotic drive systems, and propeller-based motion setups. The RS-555 is a general-purpose motor that operates efficiently in both light and moderate-load environments, making it ideal for projects requiring fast rotation and reliable performance.

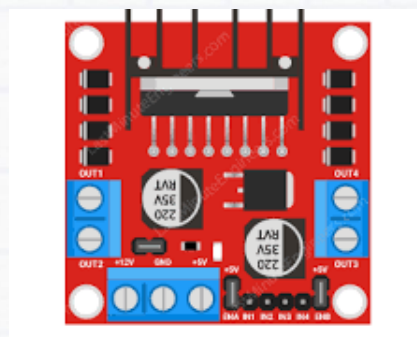
The RS-555 typically operates on a **12V DC supply**, but it supports a voltage range from **6V to 18V**, giving designers flexibility based on the torque and speed requirements. At 12V, it can deliver a **no-load speed of around 4000 RPM**, while offering a decent torque output suitable for lightweight actuation or airflow generation when paired with components like drone propellers. This motor uses **carbon brushes** and is equipped with **ball bearings** or **sleeve bearings**, depending on the variant, to enhance its life span and efficiency. The output shaft, generally **3.17 mm** in diameter, can be connected to a wide range of mechanical loads including gears, pulleys, or propellers.



Unlike servo motors that are position-controlled, the RS-555 is a **continuous rotation motor** and is controlled using **Pulse Width Modulation (PWM)** for speed variation. The direction of rotation is controlled by changing the polarity of the voltage or by using an **H-Bridge motor driver** such as the **L298N**. It typically requires a current of around **1A under load** and can go up to **2A or more** under stall conditions. For thermal management during prolonged operation, a heat sink or cooling system may be used depending on the application.

Physically, the RS-555 motor has compact dimensions with a **diameter of about 36 mm** and a **length of approximately 57 mm**, and it typically weighs around **170 grams**. With its rugged construction, consistent performance, and ease of control, the RS-555 is a preferred choice for many electronics and embedded systems projects requiring fast and reliable rotary motion.

### 3) L298N Motor Driver



The **L298N** is a popular dual-channel H-Bridge motor driver module commonly used in robotics and embedded systems to control the speed and direction of **DC motors** and **stepper motors**. It is based on the **L298N integrated circuit** manufactured by STMicroelectronics and is capable of driving two DC motors independently or one stepper motor with full control over both direction and speed.

The L298N module operates by receiving **TTL logic-level control signals** from a microcontroller (such as Arduino, ESP32, or STM32) and using them to switch high-power loads. It supports **motor supply voltages from 5V to 35V** and can deliver a continuous current of up to **2A per channel**, making it suitable for a wide range of small to medium-sized motors. The module typically includes an onboard **5V voltage regulator**, which can power the logic circuitry of the module or the microcontroller if needed. A **jumper** is used to

enable or disable this 5V regulator depending on the motor power source.

The L298N features four control input pins (IN1–IN4) to control two motors independently. Each motor has its own enable pin (ENA and ENB), which must be connected to **PWM (Pulse Width Modulation)** outputs on the microcontroller for variable speed control. By toggling the logic levels on the input pins and adjusting the PWM duty cycle on the enable pins, users can precisely control both the **speed and direction** of the connected motors.

The module also includes **built-in flyback diodes** to protect against back EMF generated by the motors, and it comes with a **large heat sink** to dissipate heat during high-load operation. It is highly durable and reusable, making it a go-to choice for prototyping and academic projects.

The L298N module typically comes in a compact rectangular PCB layout, measuring about **43 mm × 43 mm**, and includes **screw terminals** for easy motor and power connections. With its simple interface and powerful output capability, the L298N is a widely used and reliable solution for controlling motors in **robotics, automation, CNC systems, and remote-controlled vehicles**.

## Implementing the project

### Setup to install libraries

#### 1. Install the MPU6050\_light library

This library is used to interface the **MPU6050 sensor** easily with the ESP32. It simplifies accessing acceleration and gyroscopic data for real-time pitch angle calculation.

- Open **Arduino IDE**
- Go to **Sketch > Include Library > Manage Libraries...**
- In the **Library Manager**, search for “**MPU6050\_light**”
- Select the latest version by **rfetick** and click **Install**

**Note:** This library internally uses the Wire library for I2C communication. No additional I2C library is required.

## Code:

```
#include "Wire.h"

#include <MPU6050_light.h>

MPU6050 mpu(Wire);

// PID variables

unsigned long lastTime = 0;

const float sampleTime = 0.01; // 10ms control loop

float theta = 0;

float setpoint = 0; // Start at zero, can be changed to  $\pm 10$ 

float kp = 7.00;

double ki = 1.5;

float kd = 0.7;

float e = 0;

float ep = 0;

double i = 0;

float p = 0;

float d = 0;

double pid = 0;

// Motor pins

const int motor1pin1 = 13;
```



```
const int motor1pin2 = 9;

const int motor2pin1 = 18;

const int motor2pin2 = 19;

const int motor1PWM = 12;

const int motor2PWM = 20;

// Base motor speeds (minimum power to overcome friction)

const long baseSpeed = 180;

// System states

enum SystemState {STABILIZE_OFFSET, MOVE_TO_SETPOINT,
MAINTAIN_POSITION};

SystemState currentState = STABILIZE_OFFSET;

void setup() {

  Serial.begin(115200);

  // Motor control setup

  pinMode(motor1pin1, OUTPUT);

  pinMode(motor1pin2, OUTPUT);

  pinMode(motor2pin1, OUTPUT);

  pinMode(motor2pin2, OUTPUT);

  pinMode(motor1PWM, OUTPUT);

  pinMode(motor2PWM, OUTPUT);

  // Configure ESP32 PWM

  analogWriteFrequency(5000); // 5kHz PWM

  analogWriteResolution(8); // 8-bit (0-255)
```

```
// Initialize MPU6050

Wire.begin(4, 5); // SDA, SCL

Wire.setClock(400000);

byte status = mpu.begin();

while(status != 0) {

    Serial.println("MPU6050 connection failed");

    delay(1000);

}

Serial.println("Calculating offsets...");

delay(1000);

mpu.calcOffsets(); // This establishes your zero point

Serial.println("System ready");

lastTime = micros();

}

void loop() {

    mpu.update();

    // Timing control

    unsigned long now = micros();

    float deltaTime = (now - lastTime) / 1000000.0;

    if(deltaTime >= sampleTime) {

        theta = mpu.getAngleX(); // Current angle

        // PID Calculation
```

```
e = theta - setpoint;

p = kp * e;

i += ki * e * deltaTime;

i = constrain(i, -100, 100); // Anti-windup

d = kd * (e - ep) / deltaTime;

pid = p + i + d;

ep = e;

// Motor control
controlMotors(pid);

// State machine for different behaviors
switch(currentState) {

    case STABILIZE_OFFSET:

        if(abs(theta - setpoint) < 1.0) { // Within 1° of target

            currentState = MAINTAIN_POSITION;

        }

        break;

    case MOVE_TO_SETPOINT:

        if(abs(theta - setpoint) < 1.0) {

            currentState = MAINTAIN_POSITION;

        }

        break;

    case MAINTAIN_POSITION:
```



```
// Just keep maintaining the position  
break;  
  
}  
  
// Debug output  
Serial.print("Angle:");  
  
Serial.print(theta);  
  
Serial.print("\tSetpoint:");  
  
Serial.print(setpoint);  
  
Serial.print("\tPID:");  
  
Serial.println(pid);  
  
lastTime = now;  
}  
  
// Example of changing setpoint (could be triggered by button/command)  
if(Serial.available()) {  
    char cmd = Serial.read();  
  
    if(cmd == '1') setpoint = 0.0; // Move to +10°  
    if(cmd == '2') setpoint = 0.0; // Move to -10°  
    if(cmd == '0') setpoint = 0.0; // Return to zero  
    currentState = MOVE_TO_SETPOINT;  
}  
}  
  
void controlMotors(double pidOutput) {
```

```
// Set motor directions (depends on your motor wiring)

digitalWrite(motor1pin1, LOW);
digitalWrite(motor1pin2, HIGH);
digitalWrite(motor2pin1, HIGH);
digitalWrite(motor2pin2, LOW);

// Calculate motor speeds with PID correction

int motor1Speed = constrain(baseSpeed - pidOutput, 10, 255);
int motor2Speed = constrain(baseSpeed + pidOutput, 10, 255);
analogWrite(motor1PWM, motor1Speed);
analogWrite(motor2PWM, motor2Speed);
}

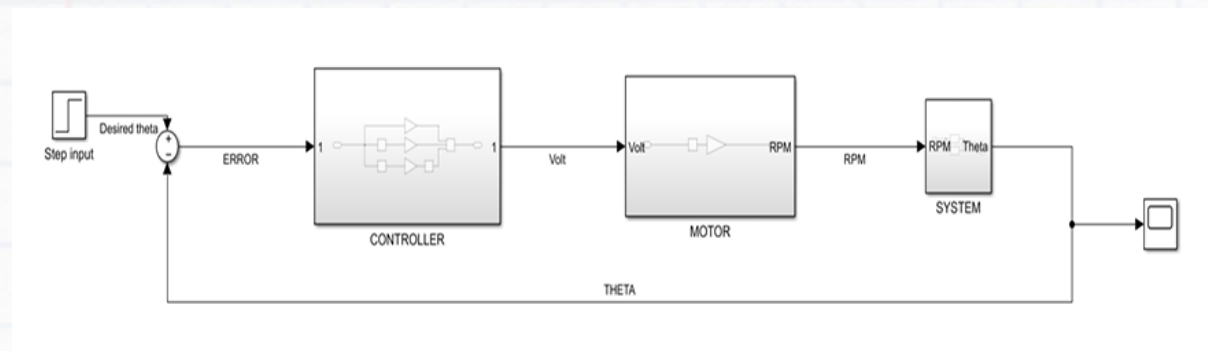
// ESP32 PWM configuration
void analogWriteFrequency(int freq) {
    ledcAttach(0, freq, 8);
    ledcAttach(1, freq, 8);
    ledcAttachChannel(motor1PWM, 0, freq, 8);
    ledcAttachChannel(motor2PWM, 1, freq, 8);
}

void analogWriteResolution(int res) {
    // Handled in frequency setup
    delay (100);
}
```

The PID controller is used to control the speed of two motors. The desired input is the angle of the MPU6050, and the actual output is the current angle of the motors. The error is calculated by subtracting the desired input from the actual output. The proportional term is then used to increase or decrease the speed of the motors based on the error. The integral term is used to compensate for any offset in the system. The derivative term is used to prevent the system from oscillating and increase response speed. **Kp=5.00; Kd=0.3; Ki=0.5.**

The tuning is done by Hit and trial.

## Control Block and Transfer Function Model



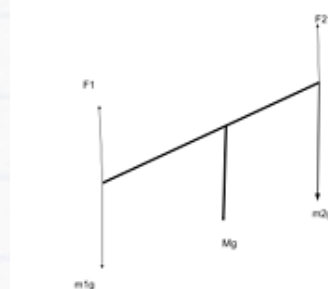
**Figure 6: Control Block Diagram**

The transfer function of this model is computed for both impulse input and external loads applied, and the system is noted to balance the pitch on its own. Final PID values of Kp, Ki, and Kd gains are noted as 6, 1.80 and 0.006 respectively. Mathematically, the two forces as reactions at the end should counteract the gravity and external force applied, taking a Freebody diagram of the system as shown:

The total moment balance about the centre:

$$I \ddot{\theta} = F_2(l/2) - F_1(l/2) - c \dot{\theta}.$$

Under the assumption of equal forces with disturbances added, such that  $\delta F$  pro] notor





## Figure 7: Forces Diagram

Mathematically, this can be simplified using a transfer function model as follows:

$$I = I_{rod} + 2 \cdot I_{motor}$$

$$= \frac{1}{12} M l^2 + 2 \cdot (m) (l/2)^2, \text{ identical motors } m_1 = m_2$$

$$= O(s)/I(s) = l/(l s^2 + c s) = l/s(l s + c)$$

where  $l$  is the length of the body, and  $c$  is the arbitrary constant of friction taken into account

Modifying this function taking  $R(s)$  as rpm,

$$O(s)/R(s) = (K/c)/s - (K\tau)/c(\tau s + c)$$

### Activities:

#### 1) PID Tuning Experiment

Manually adjust the values of  $K_p$ ,  $K_i$ , and  $K_d$  and observe the system's behaviour after each change. Note the system's response in terms of rise time, overshoot, settling time, and steady-state error. Record each configuration's effect and find the most stable and responsive combination.

**Goal:** Understand the individual and combined effects of P, I, and D on system behaviour.

#### 2) Disturbance Recovery Test

Gently apply an external disturbance to the platform (e.g., tilt or shake it by hand) and observe how the system regains balance. Evaluate how fast and accurately the PID controller corrects the deviation.

**Goal:** Test the system's robustness and real-time recovery performance.

#### 3) Data Visualization with Serial Plotter

Use **Arduino Serial Plotter** to graph the **measured pitch angle**, **setpoint**, and **error** in real time. Observe system behaviour during tuning, disturbances, and setpoint changes.

**Goal:** Visualize PID control behaviour for better understanding and debugging.

#### 4) PID vs No PID Comparison

Run the system with:

- **Only Proportional (P) control**
- **Proportional + Integral (PI) control**
- **Proportional + Derivative (PD) control**
- **Full PID control**

Compare stability, response time, and overshoot in each case.

**Goal:** Highlight the importance of each PID term through practical comparison.

### **Club Activities:**

- 1) Execute the project with load and try to maintain the stability of the system using PID algorithm.
- 2) Create and execute a similar model of pitch perfect in MATLAB/SIMULINK with different PID and observe the changes.